

Atelier Lens FastAPI 디렉터리 구조

Python + FastAPI + PostgreSQL / MVP 권장 구조

목표: 해커톤에서 과도하게 복잡하지 않으면서, OCR/Try-on/추천/결제 Provider를 나중에 교체할 수 있는 구조

1. 구조 원칙

- 도메인별 module 분리: auth, products, tryOn, cart처럼 기능 단위로 파일을 모은다.
- router → service → repository 흐름을 기본으로 한다.
- SQLAlchemy model과 Pydantic schema를 분리한다.
- OCR / Try-on / Recommendation / Payment / Social OAuth처럼 외부 구현이 바뀔 영역만 Interface + Provider로 분리한다.
- 파일 저장은 storage 모듈을 통해서만 접근한다.
- BackgroundTasks는 Router에서 직접 긴 로직을 실행하지 않고 tasks 함수를 등록한다.
- 공통 응답/예외/인증 의존성은 core, api/dependencies에 집중한다.

2. 최종 디렉터리 트리

```
atelier-lens-backend/
├── app/
│   ├── main.py
│   ├── core/
│   │   ├── config.py
│   │   ├── database.py
│   │   ├── security.py
│   │   ├── exceptions.py
│   │   ├── responses.py
│   │   ├── logging.py
│   │   ├── pagination.py
│   │   └── enums.py
│   ├── api/
│   │   ├── router.py
│   │   └── dependencies/
│   │       ├── auth.py
│   │       ├── database.py
│   │       ├── ownership.py
│   │       └── pagination.py
│   ├── modules/
│   │   ├── auth/
│   │   │   ├── router.py
│   │   │   ├── schemas.py
│   │   │   ├── service.py
│   │   │   ├── repository.py
│   │   │   └── models.py
│   │   ├── guests/
│   │   │   ├── router.py
│   │   │   ├── schemas.py
│   │   │   ├── service.py
│   │   │   ├── repository.py
│   │   │   └── models.py
│   │   └── users/
```

- router.py
- schemas.py
- service.py
- repository.py
- models.py

- products/
 - router.py
 - schemas.py
 - service.py
 - repository.py
 - models.py

- ocr/
 - router.py
 - schemas.py
 - service.py
 - resolver.py

- avatars/
 - router.py
 - schemas.py
 - service.py
 - repository.py
 - models.py

- try_on/
 - router.py
 - schemas.py
 - service.py
 - repository.py
 - models.py

- recommendations/
 - router.py
 - schemas.py
 - service.py

- coordis/
 - router.py
 - schemas.py
 - service.py
 - repository.py
 - models.py

- cart/
 - router.py
 - schemas.py
 - service.py
 - repository.py
 - models.py

- orders/
 - router.py
 - schemas.py
 - service.py
 - repository.py
 - models.py

- owned_products/
 - router.py
 - schemas.py
 - service.py
 - repository.py
 - models.py

```
├── diagnoses/
│   ├── router.py
│   ├── schemas.py
│   ├── service.py
│   ├── repository.py
│   └── models.py
├── repairs/
│   ├── router.py
│   ├── schemas.py
│   ├── service.py
│   ├── repository.py
│   └── models.py
├── stores/
│   ├── router.py
│   ├── schemas.py
│   ├── service.py
│   ├── repository.py
│   └── models.py
├── jobs/
│   ├── router.py
│   ├── schemas.py
│   ├── service.py
│   ├── repository.py
│   └── models.py
├── files/
│   ├── router.py
│   ├── schemas.py
│   ├── service.py
│   ├── repository.py
│   └── models.py
├── providers/
│   ├── ocr/
│   │   ├── base.py
│   │   └── paddle.py
│   ├── try_on/
│   │   ├── base.py
│   │   └── mock.py
│   ├── diagnosis/
│   │   ├── base.py
│   │   └── mock.py
│   ├── recommendations/
│   │   ├── base.py
│   │   └── rule_based.py
│   ├── payments/
│   │   ├── base.py
│   │   └── mock.py
│   └── social/
│       ├── base.py
│       ├── google.py
│       └── kakao.py
├── storage/
│   ├── base.py
│   ├── local.py
│   ├── validators.py
│   └── paths.py
├── tasks/
└── try_on.py
```

```

├── diagnosis.py
├── cleanup.py
├── utils/
│   ├── datetime.py
│   ├── files.py
│   ├── ids.py
│   └── text.py
├── alembic/
│   ├── versions/
│   └── env.py
├── tests/
│   ├── unit/
│   ├── integration/
│   └── conftest.py
├── scripts/
│   ├── seed.py
│   └── cleanup_once.py
├── data/                # gitignore, 실제 운영 경로는 환경변수 권장
│   ├── products/
│   ├── avatars/
│   ├── uploads/
│   ├── tryOn/
│   ├── diagnosis/
│   └── temporary/
├── .env.example
├── alembic.ini
├── pyproject.toml
├── README.md
└── .gitignore

```

3. 각 계층 역할

Router

HTTP에만 집중한다.

- URL / Method
- 인증 dependency
- request parsing
- multipart 수신
- service 호출
- response envelope 반환

금지: 직접 SQL, OCR 실행, 파일 경로 조립, 결제 상태 변경.

```

@router.post("/product-recognitions")
async def recognize_product(
    image: UploadFile,
    principal = Depends(get_guest_or_member),
    service: ProductRecognitionService = Depends(...),
):
    return await service.recognize(image, principal)

```

Service

업무 규칙을 담당한다.

```
Router
↓
Service
├── Repository
├── Provider Interface
└── StorageService
```

예: `OrderService.createOrder()` 는 가격/재고 검증, 주문 생성, `PaymentProvider` 호출, 재고 차감, `CartItem` 삭제를 조정한다.

Repository

DB 조회/저장만 담당한다.

- SQLAlchemy query
- transaction에 필요한 persistence method
- domain policy 판단 금지

Provider

교체 가능 외부 구현체.

```
OcrProvider          → PaddleOCR
TryOnProvider        → Mock → 실제 모델
DiagnosisProvider    → Mock → 실제 모델
RecommendationProvider → RuleBased → AI
PaymentProvider      → Mock → 실제 PG
SocialProvider       → Google / Kakao
```

4. core/ 상세

config.py

pydantic-settings 기반 환경변수 로딩 권장.

```
APP_ENV
DATABASE_URL
JWT_SECRET_KEY
ACCESS_TOKEN_EXPIRE_MINUTES=120
REFRESH_TOKEN_EXPIRE_DAYS=14
FILE_ROOT=/data/atelier-lens
AI_HTTP_TIMEOUT_SECONDS=60
GUEST_PHOTO_LIMIT=3
TRYON_SOURCE_TTL_MINUTES=60
TRYON_RESULT_TTL_MINUTES=180
JOB_TTL_HOURS=24
GOOGLE_CLIENT_ID
GOOGLE_CLIENT_SECRET
KAKAO_CLIENT_ID
KAKAO_CLIENT_SECRET
```

database.py

- PostgreSQL
- SQLAlchemy 2.x
- async engine + async session 권장
- driver: asyncpg
- request 단위 session dependency

security.py

- bcrypt hash/verify
- JWT encode/decode
- access 2시간
- refresh 14일
- refresh token hash/rotation

responses.py

모든 API가 동일 envelope를 사용한다.

```
ApiResponse[T]  
- success  
- data  
- error  
- meta.requestId  
- meta.pagination
```

exceptions.py

AppException + FastAPI exception handler.

```
ValidationError  
UnauthorizedError  
ForbiddenError  
NotFoundError  
ConflictError  
ProviderError
```

HTTP status와 API error code의 매핑을 한 곳에서 관리한다.

5. OCR 모듈

책임 분리

```
ocr/router.py  
↓  
ocr/service.py  
├── storage.validators  
├── PaddleOcrProvider  
└── ProductCodeResolver  
    ↓  
    ProductService
```

providers/ocr/base.py

```
class OcrProvider(Protocol):  
    async def recognize(self, image_path: str) -> OcrResult:  
        ...
```

paddle.py

- PaddleOCR PP-OCRv5 초기화
- 매 요청마다 모델 reload 금지

- OpenCV 1차 전처리
- 저신뢰/DB 미일치 시 2차 전처리 후 재실행

resolver.py

실제 MCM 품번 포맷이 미확정이므로 OCR Provider와 품번 규칙을 분리한다.

```
raw OCR texts
→ normalize whitespace/symbols
→ candidate ranking
→ ProductService.findByProductCode()
→ DB 일치값 우선
```

실제 품번 확보 후 resolver 규칙만 강화한다.

6. BackgroundTasks / Job

요청 시점

```
job = await job_service.create(...)
background_tasks.add_task(
    run_try_on_job,
    job.id,
    input_data,
)
return 202, job.id
```

Task 함수

tasks/try_on.py

```
Job pending
→ processing
→ TryOnService
→ StorageService save result
→ TryOn row save
→ Job succeeded + resultJson
```

오류:

```
Exception
→ Provider error mapping
→ Job failed
→ errorJson 저장
```

BackgroundTask에 SQLAlchemy request session 객체를 그대로 넘기지 않는다. task 내부에서 새 DB session을 생성한다.

7. 파일 저장 구조

storage/base.py

```
class StorageService(Protocol):
    async def save(...): ...
    async def open(...): ...
    async def delete(...): ...
```

MVP는 LocalStorageService.

저장 경로 예

```
{FILE_ROOT}/
├── products/{productId}/...
├── avatars/{userId}/...
├── uploads/guests/{guestSessionId}/...
├── uploads/members/{userId}/...
├── tryOn/{ownerId}/...
├── diagnosis/{registrationId}/...
└── temporary/...
```

DB에는 물리 경로와 메타데이터만 저장한다.

Validator

storage/validators.py 가 다음을 검사한다.

- 확장자
- Content-Type
- 실제 file signature
- 품번/개인/사후관리 이미지 최대 20MB
- 사후관리 MP4/MOV 최대 100MB

8. Provider Interface

Try-on

```
class TryOnProvider(Protocol):
    async def generate(self, request: TryOnProviderRequest) -> TryOnProviderResult:
        ...
```

MVP: MockTryOnProvider

추후: 실제 모델 class를 추가하고 config로 구현체 선택.

Payment

```
class PaymentProvider(Protocol):
    async def pay(self, request: PaymentRequest) -> PaymentResult:
        ...
```

MVP: MockPaymentProvider. 실제 PG가 추가돼도 OrderService 는 interface만 호출한다.

Recommendation

```
class RecommendationProvider(Protocol):
    async def recommend(self, product, candidates, limit) -> List[Recommendation]:
        ...
```

MVP: RuleBasedRecommendationProvider .

9. 도메인 간 의존성 규칙

허용되는 대표 의존성:

```
auth → users
ocr → products
avatars → files
try_on → products, coordis, jobs, files
recommendations → products
coordis → products
cart → products
orders → cart, products, owned_products
owned_products → products
diagnoses → owned_products, jobs, files
repairs → diagnoses, stores
files → core only
jobs → core only
```

상호 import를 피한다. 다른 도메인의 DB model을 직접 조작하기보다 공개된 Service 또는 Repository contract를 통해 접근한다.

10. Router 등록

app/api/router.py

```
api_router.include_router(auth.router, prefix="/auth")
api_router.include_router(guests.router)
api_router.include_router(users.router)
api_router.include_router(ocr.router)
api_router.include_router(products.router)
api_router.include_router(avatars.router)
api_router.include_router(try_on.router)
api_router.include_router(recommendations.router)
api_router.include_router(coordis.router)
api_router.include_router(cart.router)
api_router.include_router(orders.router)
api_router.include_router(owned_products.router)
api_router.include_router(diagnoses.router)
api_router.include_router(repairs.router)
api_router.include_router(stores.router)
api_router.include_router(jobs.router)
api_router.include_router(files.router)
```

main.py 는 앱 생성/미들웨어/router 등록만 한다.

11. 테스트 구조

```
tests/
|—— unit/
```

```

|   |   |— test_product_code_resolver.py
|   |   |— test_rule_based_recommendation.py
|   |   |— test_order_service.py
|   |   |— test_claim_service.py
|   |   |— test_file_validator.py
|   |— integration/
|   |   |— test_auth_api.py
|   |   |— test_product_recognition_api.py
|   |   |— test_try_on_job_api.py
|   |   |— test_cart_order_api.py
|   |   |— test_aftercare_api.py

```

최소 통합 테스트는 **핵심 데모 flow 전체**를 한 번 통과해야 한다.

```

guest 생성
→ 품번 OCR
→ 상품 조회
→ Try-on job
→ 추천
→ 회원 로그인
→ claim
→ 장바구니
→ mock 주문/결제
→ RegisteredProduct
→ 사후관리

```

12. Alembic 운영 규칙

- migration 파일은 Git merge 순서와 맞춰 직렬화
- 두 백엔드가 동시에 서로 다른 head를 만들지 않도록 PR merge 전 최신 main rebase
- 초기 개발 단계에서는 도메인 단위 migration 묶음 사용

```

001 auth_guest_store
002 product_file
003 experience
004 cart_order
005 aftercare

```

13. .gitignore 필수

```

.env
.env/
__pycache__/
.pytest_cache/
data/
*.log

```

실제 업로드/합성/진단 파일을 Git에 커밋하지 않는다.

14. 처음부터 만들 필요 없는 것

단일 서버 해커톤 MVP에서 다음은 **현재 도입하지 않는다**.

- Redis
- Celery
- Kafka/RabbitMQ

- Kubernetes
- 별도 Object Storage
- 실제 PG
- 실제 Virtual Try-on 모델
- Microservice 분리

대신 Interface와 Service 경계만 유지하여 추후 교체 가능하게 한다.

15. 최소 의존성 예시

```
fastapi
uvicorn[standard]
sqlalchemy
asyncpg
alembic
pydantic-settings
pyjwt
bcrypt
python-multipart
httpx
pillow
opencv-python-headless
paddleocr
paddlepaddle
```

실제 버전은 개발 시작 시 하나의 `pyproject.toml /lock` 전략으로 고정한다.

16. 최종 핵심

이 구조의 목적은 Clean Architecture 자체를 구현하는 것이 아니라 다음 네 가지를 지키는 것이다.

1. Router에 로직을 몰아넣지 않는다.
2. 도메인별 코드 소유권을 명확히 한다.
3. OCR/Try-on/결제/추천 구현체를 교체할 수 있다.
4. 백엔드 2명이 같은 파일을 동시에 수정하는 상황을 최소화한다.